

Kapitel 23. Typparametrisering

Innehållsförteckning

[Iden med templates](#)[Ett konkret exempel](#)[Multipla parametrar](#)

I detta avsnitt redogörs för grunderna i hantering av generiska klasser, s.k. *template-klasser*.

Iden med templates

De flesta program behöver spara data i någon form under sin körtid. Vanligen används någon form av datastruktur för detta data, t.ex. en vektor, lista, kö, stack, hashtabell el.dyl. Det är relativt vanligt att dessa datastrukturer inkapslas i en klass så att de skall vara lättare att använda och utvidga eller specialisera via ärvning. Vi kan ta som exempel den *stack* som fungerade som exempel i [Kapitel 12](#). Vi moderniserar den så att vi får en *Stack*-klass:

```
class Stack {
public:
    // exceptions
    enum Exception { Underflow };

    // konstruktor
    Stack () : Top(0) { };

    // metoder för stackmanipulation
    void push (const char Data);
    char pop ();

private:
    // ett element i stacken
    struct Element {
        // data för detta element
        char Data;
        // nästa element under detta i stacken
        Element * Previous;
    };

    // stackens topp
    Element * Top;
};

// push:a ett element på stacken
void Stack::push (const char Data) {
    // skapa nytt element
    Element * New = new Element;
    New->Data = Data;
    New->Previous = Top;

    // nytt toppelement
    Top = New;
}
```

```
// pop:a ett element från stacken
char Stack::pop () {
    // är stacken tom?
    if ( Top == 0 ) {
        // stacken tom
        throw Underflow;
    }

    // spara data som är lagret i elementet
    char Data = Top->Data;

    // spara pekare till det element som blir nytt topp-element
    Element * Tmp = Top;
    Top = Top->Previous;

    // frigör minne och återvänd
    delete Tmp;
    return Data;
}
```

Denna stack är totalt dynamisk och enkel att använda. De enda metoderna som finns är metoder för att lägga till ett element och ta bort det översta elementet. Den använder sig av en exception (se [Kapitel 20](#)) för att rapportera ett fel om man försöker poppa ett element ur en tom stack. Denna stack fungerar dock endast med data av typen `char`. Det är inte svårt att ändra om den att fungera med olika datatyper, det enda som behöver ändras är själva datatypen, all övrig kod hålls intakt. Vi kunde skapa olika versioner för olika datatyper, t.ex. `CharStack`, `IntStack` och `StringStack`. Låter det som slöseri med tid och minne, eftersom de olika klasserna är i princip identiska? Blir det inte problem med att hålla alla klasser uppdaterade om det blir ändringar?

Det hela går att sköta enklare och elegantare med *templates* (typparametrisering). Med typparametrisering menas att man gör den datatyp som en klass opererar på som en parameter. Vi kan alltså skapa en `Stack` och ge den typ vi vill ha som en parameter. Kompilatorn genererar sedan åt oss en stack som använder sig av just den datatyp vi vill använda. Vi kan således enkelt använda stackar för olika datatyper i samma program, men använda endast en version av koden.

Templates används oftast för att skapa *containers*, d.v.s. klasser som är menade att innehålla objekt av någon typ. I och med att containers vanligen är ganska generiska och olika typer inte kräver speciellt mycket olika kod (om alls) används templates i hög grad. Man kan även använda templates för att kapsla in t.ex. generiska algoritmer. Templates är det närmaste C++ kommer då det gäller att skriva kod som är typoberoende, d.v.s. där samma kod kan användas för olika datatyper. Vi kommer senare att se på en hel del olika fördefinierade templates i [Kapitel 24](#).

[Förutgående](#)
Serialisering

[Hem](#)

[Nästa](#)
Ett konkret exempel